Table des matières

Labo 02 — Le bronze de l'historien et de la télémétrie 1
Avant de commencer 2
Étape 1 — Écrire le bronze de l'historien 2
Étape 2 — Écrire le bronze de la télémétrie 4
Étape 3 — Contrôler chaque bronze contre sa source 5
Étape 4 — Lire ce qu'un fichier Parquet sait de lui-même 8
Étape 5 — Mesurer ce que le format change 8
Étape 6 — Versionner les scripts, jamais le lac 10
Dépannage 10

Labo 02 — Le bronze de l'historien et de la télémétrie

Labo · Module 2/12 — Formation Data Engineering

But : ranger l'historien et la télémétrie dans le lac, en Parquet, avec des types déclarés et un dossier par jour livré ; contrôler chaque bronze contre sa source ; mesurer ce que le format change ; garder les deux scripts dans le dépôt. Le pourquoi de chaque étape est dans `Theorie_02_DE.md`, section 7 et annexe A.

Deux fenêtres de l'Anaconda Prompt, environnement `formations` actif, depuis `Projet_Boreal` :

- la première pour le travail : le Bloc-notes, Git, et la ligne de commande DuckDB, qui lance les scripts (`duckdb -f`) et chronomètre les requêtes (`duckdb -c`) ;
- la seconde, à partir de l'étape 3, pour l'interface web de DuckDB (`duckdb -ui`) : les requêtes de contrôle.

Liste de contrôle :

- ☐ Dossier du lac créé : `plateforme\lac\bronze\usine`
- ☐ `bronze_historien.sql` lancé : 275 436 lignes des deux côtés, relancé à l'identique
- ☐ `bronze_telemetrie.sql` lancé : 158 146 messages des deux côtés
- ☐ Bronzes contrôlés : 92 et 843 jours, 0 écart ; 321 vides des deux côtés
- ☐ Parquet lu de l'intérieur : schéma, statistiques, taille des fichiers

© 2026 iAlexMG. Tous droits réservés.

- ☐ Gains mesurés : trois temps sur l'historien, `Scanning Files: 1/92`, deux temps sur la télémétrie
- ☐ Les deux scripts validés dans le dépôt `01_de`, `git status` propre

Avant de commencer

Anaconda Prompt, depuis `Projet_Boreal` :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
dir 01_de\sql
REM Attendu : un fichier, premier_regard.sql.
```

Les lignes `REM` sont des commentaires : l'Anaconda Prompt les ignore. Si un résultat diffère, finir d'abord le Labo 01 (étapes 1 et 5).

Étape 1 — Écrire le bronze de l'historien

Théorie : `Theorie_02_DE.md`, annexe A.1.

1.1 Créer le script. Anaconda Prompt, depuis `Projet_Boreal` :

```
notepad 01_de\sql\bronze_historien.sql
```

Le Bloc-notes propose de créer le fichier : accepter. Coller, enregistrer, fermer :

```
-- 01_de/sql/bronze_historien.sql : l'historien, du CSV au Parquet partitionné par jour livré.
-- Lancement, depuis Projet_Boreal : duckdb -f 01_de/sql/bronze_historien.sql

COPY (
    SELECT
        TagName,
        Timestamp,
        Value,
        Quality,
        CAST(REPLACE(PARSE_FILENAME(filename, true), 'historien_', '') AS DATE) AS jour
    FROM read_csv(
        'sources/usine/historien/*.csv',
        header = true,
        filename = true,
```


© 2026 iAlexMG. Tous droits réservés.

```
        columns = {'TagName': 'VARCHAR', 'Timestamp': 'TIMESTAMP', 'Value': 'DOUBLE', 'Quality': 'INTEGER'}
    )
) TO 'plateforme/lac/bronze/usine/historien'
  (FORMAT parquet, COMPRESSION zstd, PARTITION_BY (jour), OVERWRITE);

-- Contrôle. Attendu : 275 436 des deux côtés.
SELECT
    (SELECT COUNT(*) FROM 'sources/usine/historien/*.csv') AS lignes_csv,
    (SELECT COUNT(*) FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet') AS lignes_parquet;
```

1.2 Erreur voulue : lancer le script avant que le lac existe.

```
duckdb -f 01_de/sql/bronze_historien.sql
```

Attendu : une erreur, `IO Error: Failed to create directory "plateforme/lac/bronze/usine/historien": Le chemin d'accès spécifié est introuvable.` Rien n'est écrit.

À retenir — `COPY` crée le dernier dossier du chemin, jamais ceux du dessus. Ici, il aurait créé `historien`, mais `plateforme\lac\bronze\usine` n'existe pas encore. Tout script qui écrit dans un lac neuf, sur un autre poste ou dans un autre projet, tombe sur ce message. On le provoque ici pour le reconnaître : il signale un dossier parent manquant, pas un script faux.

1.3 Créer le dossier du lac, puis relancer.

```
mkdir plateforme\lac\bronze\usine
duckdb -f 01_de/sql/bronze_historien.sql
```

Attendu : après quelques secondes, `lignes_csv` 275436 et `lignes_parquet` 275436.

1.4 Relancer le script, pour prouver qu'il se rejoue sans rien doubler.

```
duckdb -f 01_de/sql/bronze_historien.sql
```

Attendu : le même contrôle, 275436 et 275436 : rien n'a doublé. Un chargement se relance après une panne ou une livraison reprise : s'il doublait les lignes à chaque passage, on ne pourrait plus le relancer sans risque. Le Cours 03 fait de cette propriété une règle, l'idempotence.

1.5 Erreur voulue : écrire dans le bronze sans `OVERWRITE`.

```
duckdb -c "COPY (SELECT 1 AS x, DATE '2025-10-28' AS jour) TO 'plateforme/lac/bronze/usine/historien' (FORMAT parquet,
PARTITION_BY (jour))"
```

Attendu : une erreur, `IO Error: Directory "plateforme/lac/bronze/usine/historien" is not empty! Enable OVERWRITE option to overwrite files`. Rien n'est écrit : les 92 fichiers du bronze restent intacts.

© 2026 iAlexMG. Tous droits réservés.

À retenir — sans option, `COPY` refuse d'écrire dans un dossier qui contient déjà des fichiers : il protège le bronze d'un écrasement par mégarde. `OVERWRITE` lève cette protection en vidant tout le dossier avant d'écrire. C'est lui qui a rendu 1.4 identique à 1.3, et c'est pourquoi le script réécrit toujours le bronze en entier, jamais à moitié.

Étape 2 — Écrire le bronze de la télémétrie

Théorie : `Theorie_02_DE.md`, annexe A.2.

2.1 Créer le second script :

```
notepad 01_de\sql\bronze_telemetrie.sql
```

Accepter la création. Coller, enregistrer, fermer :

```
-- 01_de/sql/bronze_telemetrie.sql : la télémétrie, du JSON lines au Parquet imbriqué, par jour de réception.
-- Lancement, depuis Projet_Boreal : duckdb -f 01_de/sql/bronze_telemetrie.sql

COPY (
    SELECT
        sujet,
        recu_le,
        qos,
        message,
        CAST(REPLACE(PARSE_FILENAME(filename, true), 'sorties_', '') AS DATE) AS jour
    FROM read_json(
        'sources/telemetrie/*.jsonl',
        format = 'newline_delimited',
        filename = true,
        columns = {
            'sujet': 'VARCHAR',
            'recu_le': 'VARCHAR',
            'qos': 'INTEGER',
            'message': 'STRUCT(
                message_id VARCHAR, sernr VARCHAR, micrologiciel VARCHAR,
                debut VARCHAR, fin VARCHAR,
                distance_km DOUBLE, heures_moteur DOUBLE,
                regime_moyen INTEGER, regime_max INTEGER,
                temp_moteur_max DOUBLE, temperature_moteur_max DOUBLE,
                temp_exterieure DOUBLE, tension_batterie_min DOUBLE,
                position STRUCT(lat DOUBLE, lon DOUBLE),
                codes VARCHAR[])'
        }
```


© 2026 iAlexMG. Tous droits réservés.

```
    )
) TO 'plateforme/lac/bronze/telemetrie'
  (FORMAT parquet, COMPRESSION zstd, PARTITION_BY (jour), OVERWRITE);

-- Contrôle. Attendu : 158 146 des deux côtés.
SELECT
    (SELECT COUNT(*) FROM read_json('sources/telemetrie/*.jsonl', format = 'newline_delimited',
                                    columns = {'sujet': 'VARCHAR'})) AS messages_json,
    (SELECT COUNT(*) FROM 'plateforme/lac/bronze/telemetrie/*/*.parquet') AS messages_parquet;
```

2.2 Lancer le script.

```
duckdb -f 01_de/sql/bronze_telemetrie.sql
```

Attendu : `messages_json` 158146 et `messages_parquet` 158146.

Étape 3 — Contrôler chaque bronze contre sa source

Théorie : `Theorie_02_DE.md`, annexe A.3.

Ouvrir une seconde Anaconda Prompt, activer `formations`, se placer dans `Projet_Boreal`, lancer l'interface :

```
duckdb -ui
```

Laisser cette fenêtre ouverte : la fermer arrête l'interface. Elle rouvre le dernier carnet, celui du Labo 01 : le laisser tel quel, en créer un neuf (+ à côté de Notebooks) et le renommer « Labo 02 » (double-clic sur son titre, puis Entrée). Une requête par cellule, Ctrl+Entrée pour l'exécuter.

L'historien

3.1 Peser le bronze.

```
-- Attendu : 92 fichiers, 1 468 497 octets.
SELECT COUNT(*) AS fichiers, SUM(size) AS octets
FROM read_blob('plateforme/lac/bronze/usine/historien/*/*.parquet');
```

3.2 Comparer les lignes jour par jour, source contre bronze.

```
-- Attendu : 92 jours, 0 écart.
WITH livre AS (
  SELECT CAST(REPLACE(PARSE_FILENAME(filename, true), 'historien_', '') AS DATE) AS jour,
         COUNT(*) AS lignes
  FROM read_csv('sources/usine/historien/*.csv', filename = true)
  GROUP BY ALL
),
```


© 2026 iAlexMG. Tous droits réservés.

```
bronze AS (
  SELECT jour, COUNT(*) AS lignes
  FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet'
  GROUP BY ALL
)
SELECT COUNT(*) AS jours,
       COUNT(*) FILTER (WHERE livre.lignes IS DISTINCT FROM bronze.lignes) AS ecarts
FROM livre FULL JOIN bronze USING (jour);
```

3.3 Voir les types du bronze.

```
-- Attendu : 5 lignes. TagName VARCHAR, Timestamp TIMESTAMP, Value DOUBLE, Quality INTEGER, jour DATE.
DESCRIBE SELECT * FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet';
```

3.4 Vérifier que les valeurs vides ont survécu.

```
-- Attendu : 321 et 321.
SELECT
  (SELECT COUNT(*) FILTER (WHERE Value IS NULL)
   FROM read_csv('sources/usine/historien/*.csv', header = true,
                 columns = {'TagName': 'VARCHAR', 'Timestamp': 'TIMESTAMP', 'Value': 'DOUBLE', 'Quality': 'INTEGER'}))
   AS vides_csv,
  (SELECT COUNT(*) FILTER (WHERE Value IS NULL)
   FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet')
   AS vides_bronze;
```

3.5 Retrouver les valeurs de départ dans la partition de leur livraison.

```
-- Attendu : 58 lignes datées du 30 septembre dans jour=2025-10-01, une par étiquette.
SELECT COUNT(*) AS lignes, COUNT(DISTINCT TagName) AS etiquettes
FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet'
WHERE jour = DATE '2025-10-01' AND Timestamp < TIMESTAMP '2025-10-01';
```

La télémétrie

3.6 Peser le bronze.

```
-- Attendu : 843 fichiers, 10 817 501 octets.
SELECT COUNT(*) AS fichiers, SUM(size) AS octets
FROM read_blob('plateforme/lac/bronze/telemetrie/*/*.parquet');
```

3.7 Comparer les messages jour par jour, source contre bronze.

```
-- Attendu : 843 jours, 0 écart.
WITH livre AS (
```


© 2026 iAlexMG. Tous droits réservés.

```
  SELECT CAST(REPLACE(PARSE_FILENAME(filename, true), 'sorties_', '') AS DATE) AS jour,
         COUNT(*) AS messages
  FROM read_json('sources/telemetrie/*.jsonl', format = 'newline_delimited', filename = true,
                 columns = {'sujet': 'VARCHAR'})
  GROUP BY ALL
),
bronze AS (
  SELECT jour, COUNT(*) AS messages
  FROM 'plateforme/lac/bronze/telemetrie/*/*.parquet'
  GROUP BY ALL
)
SELECT COUNT(*) AS jours,
       COUNT(*) FILTER (WHERE livre.messages IS DISTINCT FROM bronze.messages) AS ecarts
FROM livre FULL JOIN bronze USING (jour);
```

3.8 Voir la forme des messages dans le bronze.

```
-- Attendu : 5 lignes. sujet VARCHAR, recu_le VARCHAR, qos INTEGER, jour DATE, et message
-- STRUCT(message_id VARCHAR, sernr VARCHAR, …, "position" STRUCT(lat DOUBLE, lon DOUBLE), codes VARCHAR[]).
DESCRIBE SELECT * FROM 'plateforme/lac/bronze/telemetrie/*/*.parquet';
```

3.9 Compter les deux noms de la température du moteur.

```
-- Attendu : 88 678 avec temp_moteur_max, 69 468 avec temperature_moteur_max, 158 146 en tout.
SELECT COUNT(message.temp_moteur_max) AS avant_2_0,
       COUNT(message.temperature_moteur_max) AS depuis_2_0,
       COUNT(*) AS messages
FROM 'plateforme/lac/bronze/telemetrie/*/*.parquet';
```

3.10 Lire les horodatages tels que reçus.

```
-- Attendu : 2025-12-15T00:25:22-05:00 et 2025-12-14T09:53:00-05:00.
SELECT recu_le, message.debut
FROM 'plateforme/lac/bronze/telemetrie/*/*.parquet'
WHERE jour = DATE '2025-12-15'
ORDER BY recu_le
LIMIT 1;
```

3.11 Compter les décalages horaires gardés dans `debut`.

```
-- Attendu : 8 lignes, de -02:30 (4 420 messages) à -08:00 (4 878) ; le plus fréquent, -04:00 (65 875).
SELECT RIGHT(message.debut, 6) AS decalage, COUNT(*) AS messages
FROM 'plateforme/lac/bronze/telemetrie/*/*.parquet'
```


© 2026 iAlexMG. Tous droits réservés.

```
GROUP BY ALL
ORDER BY decalage;
```

Étape 4 — Lire ce qu'un fichier Parquet sait de lui-même

Théorie : `Theorie_02_DE.md`, annexe A.4.

Toujours dans le carnet.

4.1 Lire le schéma qu'un fichier de l'historien porte en lui.

```
-- Attendu : 5 lignes. duckdb_schema, la racine ; TagName BYTE_ARRAY (UTF8) ; Timestamp INT64
-- (TIMESTAMP_MICROS) ; Value DOUBLE ; Quality INT32 (INT_32).
SELECT name, type, converted_type
FROM parquet_schema('plateforme/lac/bronze/usine/historien/jour=2025-10-28/*.parquet');
```

4.2 Lire ses statistiques.

```
-- Attendu : 4 lignes, un seul groupe de 4 106 lignes, en ZSTD. TagName de ESS.BANC.REGIME à
-- UTL.SEC.POINT_ROSEE ; Timestamp de 2025-10-27 00:00:00 à 2025-10-28 23:58:00. Octets compressés :
-- TagName 3 260, Timestamp 4 529, Value 12 598, Quality 94.
SELECT path_in_schema, row_group_num_rows, compression, stats_min, stats_max,
       total_compressed_size, total_uncompressed_size
FROM parquet_metadata('plateforme/lac/bronze/usine/historien/jour=2025-10-28/*.parquet');
```

4.3 Mesurer la taille des fichiers de la télémétrie.

```
-- Attendu : 843 fichiers, 13 Ko en moyenne, de 3 227 à 47 736 octets.
SELECT COUNT(*) AS fichiers, ROUND(AVG(size) / 1000) AS ko_moyens,
       MIN(size) AS plus_petit, MAX(size) AS plus_gros
FROM read_blob('plateforme/lac/bronze/telemetrie/*/*.parquet');
```

Étape 5 — Mesurer ce que le format change

Théorie : `Theorie_02_DE.md`, annexe A.5.

Retour dans la première Anaconda Prompt. Chaque mesure pose la même question à la source, puis au bronze : la réponse ne change pas, le temps oui. L'écart vient du format et des types.

5.1 Chronométrer trois lectures de l'historien, pour la même question : la vibration maximale du compresseur 2.

© 2026 iAlexMG. Tous droits réservés.

Commande	Ce que DuckDB lit	Ce qu'il fait avant de répondre	Chez l'auteur
1. CSV, types devinés	les 92 CSV, 13,7 Mo de texte	il devine les types sur un échantillon, puis lit et convertit chaque ligne	3,1 s
2. CSV, types déclarés	les mêmes 92 CSV	il lit et convertit chaque ligne, sans rien deviner	0,6 s
3. Bronze Parquet	les 92 fichiers du bronze, 1,5 Mo	il ne lit que `TagName` et `Value`, deux colonnes sur quatre, déjà typées et compressées	0,02 s

```
duckdb -cmd ".timer on" -c "SELECT MAX(Value) FROM 'sources/usine/historien/*.csv' WHERE TagName = 'UTL.CMP2.VIBRATION'"
duckdb -cmd ".timer on" -c "SELECT MAX(Value) FROM read_csv('sources/usine/historien/*.csv', header = true, columns = {'TagName': 'VARCHAR', 'Timestamp': 'TIMESTAMP', 'Value': 'DOUBLE', 'Quality': 'INTEGER'}) WHERE TagName = 'UTL.CMP2.VIBRATION'"
duckdb -cmd ".timer on" -c "SELECT MAX(Value) FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet' WHERE TagName = 'UTL.CMP2.VIBRATION'"
```

Où : `-cmd ".timer on"` fait afficher la durée sous le résultat, `Run Time (s): real …`. Seul le chiffre après `real` compte : le temps écoulé.

Attendu : 9.23 trois fois, et trois temps qui fondent comme ceux du tableau. Les vôtres seront autres ; l'ordre de grandeur, non.

5.2 Voir l'élagage des partitions : un filtre sur `jour` n'ouvre qu'un fichier sur 92.

```
duckdb -c "EXPLAIN ANALYZE SELECT COUNT(*) FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet' WHERE jour = DATE '2025-10-28'"
```

Attendu, dans le dernier bloc du plan, celui du `TABLE_SCAN` : `File Filters: (jour = '2025-10-28'::DATE)`, `Scanning Files: 1/92`, `Total Files Read: 1` et `4,106 rows`.

5.3 Chronométrer deux lectures de la télémétrie, pour la même question : le nombre de véhicules qui publient.

Commande	Ce que DuckDB lit	Ce qu'il fait avant de répondre	Chez l'auteur
1. JSON lines	les 884 fichiers de l'archive, 70,6 Mo de texte	il décode chaque ligne, fichier par fichier, pour en tirer `message.sernr`	0,7 s
2. Bronze Parquet	les 843 fichiers du bronze, 10,8 Mo	il ne lit que la colonne `message`, déjà typée, mais ouvre 843 petits fichiers	0,15 s

```
duckdb -cmd ".timer on" -c "SELECT COUNT(DISTINCT message.sernr) AS vehicules FROM read_json('sources/telemetrie/*.jsonl', union_by_name = true)"
```


© 2026 iAlexMG. Tous droits réservés.

```
duckdb -cmd ".timer on" -c "SELECT COUNT(DISTINCT message.sernr) AS vehicules FROM
'plateforme/lac/bronze/telemetrie/*/*.parquet'"
```

Attendu : 6759 deux fois. Le bronze gagne encore, mais bien moins que sur l'historien : ouvrir 843 fichiers de 13 Ko coûte cher. C'est le piège des petits fichiers.

Étape 6 — Versionner les scripts, jamais le lac

Théorie : `Theorie_02_DE.md`, annexe A.6.

Toujours dans la première Anaconda Prompt, depuis `Projet_Boreal` :

```
git -C 01_de add sql\bronze_historien.sql sql\bronze_telemetrie.sql
git -C 01_de commit -m "Cours 02 : bronze de l'historien et de la télémétrie en Parquet"
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

Pour tout fermer : `.exit` à l'invite `D` de la seconde Anaconda Prompt, puis fermer l'onglet du navigateur.

Dépannage

Symptôme	**Correctif**
`No files found that match the pattern`	DuckDB n'a pas été lancé dans `Projet_Boreal` : `.exit` au besoin, se placer dans `Projet_Boreal`, relancer
`IO Error: Failed to open file "01_de/sql/bronze_historien.sql"`	le Bloc-notes l'a enregistré en `bronze_historien.sql.txt` (`dir 01_de\sql` le montre) : `ren 01_de\sql\bronze_historien.sql.txt bronze_historien.sql`
`mkdir` répond que la syntaxe est incorrecte	des `/` au lieu de `\` : `mkdir plateforme\lac\bronze\usine`
1.2 ne donne pas d'erreur	le dossier du lac existe déjà (labo refait) : passer à 1.4
`IO Error: Failed to create directory` en 1.3	le `mkdir` de 1.3 n'a pas été fait : le faire, puis relancer
`Directory "…" is not empty!` au lancement d'un script	`OVERWRITE` a disparu du script : la remettre dans la parenthèse du `COPY`
Le contrôle d'un script rend deux nombres différents	les sources ont été regénérées depuis, ou le script s'est arrêté en route : relancer le script

© 2026 iAlexMG. Tous droits réservés.

Symptôme	Correctif
Le contrôle de l'historien rend 2 133 145 lignes côté CSV	le générateur a tourné avec `--historien complet` : relancer le script si c'est voulu ; sinon, regénérer sans option (dernière ligne)
`Binder Error: Referenced table "message" not found!`	l'archive a été lue sans déclarer sa forme : déclarer les colonnes, comme le script, ou `union_by_name = true` (Labo 01, 4.6)
`Conversion Error` sur `Timestamp` ou `Value`	un fichier de l'historien a été modifié à la main : regénérer les sources (dernière ligne)
Le bronze existe, mais `01_de\sql` n'a pas les deux scripts	le code a été tapé dans DuckDB au lieu d'un fichier : créer les scripts avec `notepad` (1.1, 2.1) et les relancer
La cellule n'affiche qu'un seul résultat	elle contient plusieurs requêtes : une requête par cellule
La page ne répond plus	la seconde Anaconda Prompt a été fermée : y relancer `duckdb -ui` depuis `Projet_Boreal`
Un compte ne donne pas le chiffre attendu	relancer le générateur, depuis `Data_Domain` : `python _univers/boreal/generer_boreal.py --projet Projet_Boreal`, puis les deux scripts

© 2026 iAlexMG. Tous droits réservés.